COMP 8005
Assignment 1
Report

Winston Nguyen
A01297231
Feb 1st, 2026

Purpose

This program will attempt to crack a password that is hashed using one of the following algorithms: MD5, SHA-256, SHA-512, bcrypt, and yescrypt

Requirements

Task	Status
Crack the password and display results	Fully implemented

Platforms

controller.py and worker.py has been tested on:

- Ubuntu 2023.10

Language

- Python

Documents

- Design
- Testing

Findings

Design

Overall Architecture

The architecture of this program is a simple single-threaded controller/worker architecture.

Controller/Worker Responsibilities

Controller

The controller is responsible for setting up a server, and to parse out the shadow file information from the given user. The controller will parse out the: user, hash algorithm, salts, and the hashed password. This information will be passed onto the worker that connects with it to crack the password.

Worker

The worker is responsible for connecting with the server/controller and waiting for a job. After receiving the password information, the worker will then use the hash algorithm and password information to use a brute-force method to crack the password, and will return the results back to the controller.

Communication Protocol

The controller and worker communicate through a TCP socket connection. The controller will start the TCP socket, and await a connection. The worker will then connect, and await for a response from the server where it will receive the cracking information. After cracking the password, the worker finishes by sending a final response with all the necessary information to the controller.

Partitioning Strategy

As the worker is single threaded, there is no partitioning in the way the worker cracks the password, and it simply uses brute-force to exhaust all possible options.

Implementation

Languages and Libraries

Languages

This program was created using Python 3.13.

Libraries

socket: Create the TCP connection
pickle: Dump the communication information for the connection
argparse: Parse command line arguments
datetime: Calculates run time lengths
Ipaddress: Check if an IP address is valid
bcrypt: Hash algorithm for bcrypt
crypt: Hash algorithm for yescrypt, SHA-256, SHA-512, MD5

itertools: Iterates through a list of characters
string: Creates list of characters to iterate through

Experimental Results

Algorithm Run Times

All runs were done using the following password: abc, A2#, 456

MD5	Min (seconds)	Max (seconds)	Mean (seconds)
Controller Parsing Time	0.007092	0.007751	0.007312
Job Dispatch Latency	0.016363	0.1947231	0.0787917
Cracking Time	0.571732	32.559221	16.26098267
Return Latency	0.02125	0.205381	0.08690933333
Total end-to-end Runtime	3.561421	34.75676	19.77685133

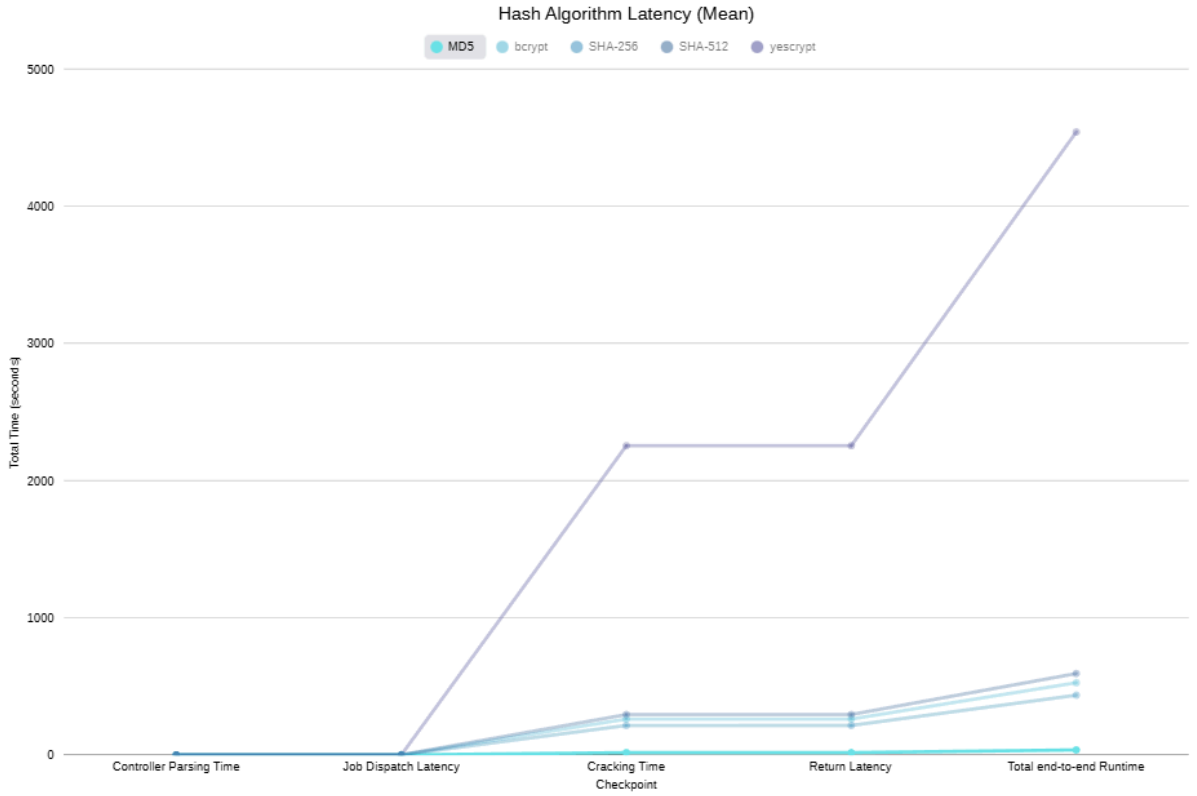
bcrypt	Min (seconds)	Max (seconds)	Mean (seconds)
Controller Parsing Time	0.00702	0.007865	0.007477
Job Dispatch Latency	0.006521	0.196824	0.071511
Cracking Time	9.178933	523.99748	260.9191297
Return Latency	0.024409	0.331639	0.1865556667
Total end-to-end Runtime	12.165712	526.191424	264.9991107

SHA-256	Min (seconds)	Max (seconds)	Mean (seconds)
Controller Parsing Time	0.006996	0.00858	0.007858666667
Job Dispatch Latency	0.016035	0.196985	0.07837366667
Cracking Time	7.045302	432.015387	214.4495927
Return Latency	0.004564	0.204128	0.079737
Total end-to-end Runtime	8.56452	443.861367	220.5980913

SHA-512	Min (seconds)	Max (seconds)	Mean (seconds)
Controller Parsing Time	0.004973	0.009052	0.006729666667
Job Dispatch Latency	0.070059	0.196383	0.1127586667
Cracking Time	9.264756	614.28684	294.114491
Return Latency	0.071774	0.203022	0.117221
Total end-to-end Runtime	11.079179	620.524745	298.7454607

yescrypt	Min (seconds)	Max (seconds)	Mean (seconds)
Controller Parsing Time	0.004854	0.007991	0.006171
Job Dispatch Latency	0.042938	0.196443	0.09989066667
Cracking Time	73.082301	4520.129503	2253.490109
Return Latency	0.082694	0.204211	0.1396143333
Total end-to-end Runtime	74.996658	4575.461873	2286.507314

Graph



Analysis

For every algorithm, the bottlenecks were the cracking time. The latency between sending jobs or returning data, or even parsing the shadow file are miniscule compared to the time it takes to crack the password single-threaded using brute force. Runs using simple cases, with characters that came early in the alphabet lowercase, showed significantly shorter run times compared to passwords with a mix of letters, numbers, and special characters. The yescrypt hash algorithm also made the cracking time substantially longer than the others, while MD5 was the fastest.